

# FALCO ROVER

## Autonomous Indoor Navigation System

*GPS-Denied Environment / Multi-Sensor Fusion / ROS2 Humble*

**Prepared by: Shaan Siddiqui**

Platform: ROS2 Humble | Gazebo Classic | Ubuntu 22.04

Submission: Falco Robotics

## Contents

|                                  |           |
|----------------------------------|-----------|
| <b>1. Introduction</b>           | <b>3</b>  |
| <b>2. System Architecture</b>    | <b>4</b>  |
| <b>3. Navigation Algorithm</b>   | <b>6</b>  |
| <b>4. Sensor Specifications</b>  | <b>9</b>  |
| <b>5. Multi-Sensor Fusion</b>    | <b>12</b> |
| <b>6. ROS2 Node Architecture</b> | <b>14</b> |
| <b>7. Assumptions</b>            | <b>19</b> |
| <b>8. Custom Contributions</b>   | <b>20</b> |
| <b>9. Simulation Results</b>     | <b>22</b> |
| <b>10. Conclusion</b>            | <b>25</b> |

## 1. Introduction

The Falco Rover is a fully autonomous ground robot designed to navigate inside closed, GPS-denied indoor environments. The system uses a combination of onboard sensors — an RP LiDAR A1M8, an Intel RealSense D435 depth camera, and an IMU — fused together to provide robust localisation, obstacle detection, and path planning without any GPS signal or continuous user input.

The system is built on the ROS2 Humble framework and simulated in Gazebo Classic. It implements a complete autonomous navigation pipeline: environment mapping via SLAM, localisation via AMCL, path planning via Nav2, and multi-sensor obstacle avoidance using both LiDAR and depth camera data fused into a combined costmap.

Voice announcements and LED status indicators provide real-time feedback on the robot's navigation state, implemented as custom ROS2 nodes. The entire system is launched from a single command using a custom master launch file.

## 2. System Architecture

The Falco Rover is based on the TurtleBot3 Waffle Pi platform, extended with a custom URDF model (falco\_rover) that adds a depth camera link and Gazebo simulation plugins for all sensors. The robot uses a differential drive configuration with two powered wheels and two passive casters.

### 2.1 Hardware Components

| Component                 | Role                                         | ROS2 Topic                                  |
|---------------------------|----------------------------------------------|---------------------------------------------|
| RP LiDAR A1M8             | 360° environment mapping, obstacle detection | /scan                                       |
| Intel RealSense D435      | 3D depth perception, close-range obstacles   | /depth_camera/points,<br>/depth_camera/scan |
| IMU Sensor (MPU6050)      | Orientation and acceleration measurement     | /imu                                        |
| Differential Drive Motors | Robot locomotion                             | /cmd_vel, /odom                             |
| RPi (simulated by CPU)    | Central computation and ROS2 node execution  | —                                           |
| Voice System (espeak)     | Audio status announcements                   | Navigation action topics                    |
| LED Indicator             | Visual robot status (GREEN/RED/YELLOW)       | Navigation action topics                    |

## 2.2 Software Stack

| Component                | Function                                                                         |
|--------------------------|----------------------------------------------------------------------------------|
| ROS2 Humble              | Robot operating system framework — middleware for all node communication         |
| Gazebo Classic           | Physics simulation with sensor plugins (LiDAR, depth camera, IMU, diff drive)    |
| SLAM Toolbox             | Simultaneous Localisation and Mapping — builds occupancy grid map from LiDAR     |
| AMCL                     | Adaptive Monte Carlo Localisation — probabilistic pose estimation on saved map   |
| Nav2                     | Global path planning (A*), local obstacle avoidance (DWB), recovery behaviours   |
| EKF (robot_localization) | Extended Kalman Filter — fuses IMU and odometry for accurate position estimate   |
| pointcloud_to_laserscan  | Converts 3D depth camera pointcloud to 2D laser scan for Nav2 costmap            |
| voice_commander.py       | Custom node — announces navigation events via espeak text-to-speech              |
| indicator.py             | Custom node — publishes LED status (GREEN=goal reached, RED=idle, YELLOW=moving) |

### 3. Navigation Algorithm

The navigation algorithm operates in two phases: a one-time mapping phase and a repeatable autonomous navigation phase.

#### 3.1 Phase 1 — Environment Mapping (SLAM)

Before autonomous navigation can begin, the robot builds a map of the environment using Simultaneous Localisation and Mapping (SLAM). This is performed once and the map is saved for reuse.

- The SLAM Toolbox node subscribes to /scan (LiDAR data at 4.7 Hz)
- As the robot is manually driven around the environment via teleop, the LiDAR scan data is incrementally processed
- SLAM correlates successive scans using scan matching to estimate robot motion and build an occupancy grid
- Each cell in the occupancy grid is marked as: occupied (obstacle), free (navigable), or unknown
- The completed map is saved as my\_map.yaml + my\_map.pgm for reuse in navigation phase

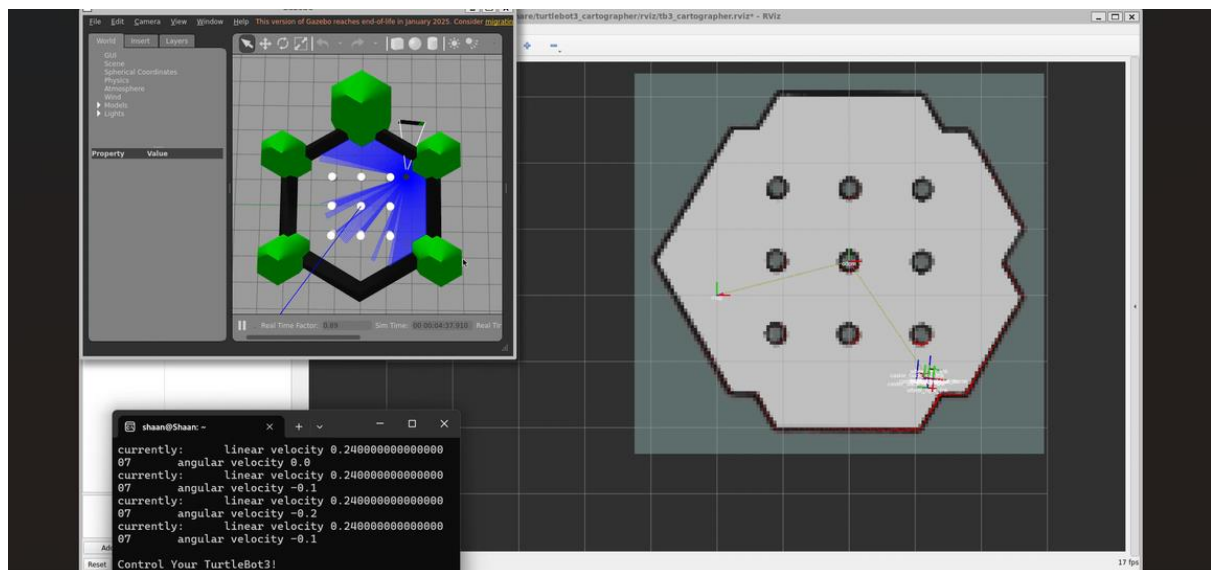


Figure 1: SLAM Occupancy Grid Map — Built using SLAM Toolbox in Gazebo

## 3.2 Phase 2 — Autonomous Navigation

The core navigation algorithm follows a multi-layer decision pipeline:

### Step 1: Sensor Fusion via EKF

The Extended Kalman Filter (`ekf_filter_node`) continuously fuses two data sources — wheel odometry (`/odom`) and IMU data (`/imu`) — to produce a filtered, drift-corrected pose estimate published on `/odometry/filtered`. This is the robot's best estimate of its position before map-based localisation.

### Step 2: Depth Camera Integration

The `pointcloud_to_laserscan_node` subscribes to `/depth_camera/points` (3D pointcloud at 30 Hz from the RealSense D435 plugin) and slices it into a 2D laser scan at a configurable height range (0.0m to 0.5m). This converted scan is published on `/depth_camera/scan` and fed into Nav2's obstacle layer alongside the LiDAR scan, enabling detection of obstacles that the 2D LiDAR may miss.

### Step 3: Map-Based Localisation via AMCL

AMCL (Adaptive Monte Carlo Localisation) uses a particle filter to estimate the robot's pose on the pre-built map. It subscribes to `/scan` and `/map`, and uses particle weights — computed by comparing current LiDAR readings against the expected map — to converge on the most probable robot pose. The result is published as `/amcl_pose` and broadcast on the `/tf` tree (`map` → `odom` → `base_link` chain).

### Step 4: Costmap Construction

Two costmaps are maintained simultaneously:

- Global costmap — static map layer + obstacle layer (LiDAR + depth camera) + inflation layer. Used for global path planning across the full environment.
- Local costmap — rolling window (3m x 3m) around the robot. Voxel layer + obstacle layer subscribing to both `/scan` and `/depth_camera/scan`. Used for real-time obstacle avoidance.

### **Step 5: Global Path Planning (A\*)**

When a navigation goal is received on `/goal_pose`, the `planner_server` uses the NavFn planner (A\* algorithm) on the global costmap to compute an optimal path from the robot's current pose to the goal. The path avoids all known obstacles with the inflation radius applied as a safety margin.

### **Step 6: Local Obstacle Avoidance (DWB)**

The `controller_server` executes the DWB (Dynamic Window Based) local planner, which continuously samples velocity commands in the robot's dynamic window, simulates trajectories, scores them against critics (PathAlign, GoalAlign, ObstacleCost, RotateToGoal), and publishes the best velocity command on `/cmd_vel` at 10 Hz. This allows the robot to follow the global plan while reactively avoiding newly detected obstacles.

### **Step 7: Recovery Behaviours**

If the robot becomes stuck (no progress detected by the progress checker), the `behaviour_server` triggers recovery actions in sequence: Spin (rotate in place to re-acquire LiDAR context), BackUp (reverse away from obstacle), Wait (pause and retry). These allow the robot to self-recover without user intervention.

### **Step 8: Goal Notification**

On reaching the goal, the `bt_navigator` signals completion. The `voice_commander` node announces the result via `espeak` and the `indicator_node` sets LED status to GREEN. During navigation, the indicator shows YELLOW; when idle or failed, RED.



## 4. Sensor Specifications

### 4.1 RP LiDAR A1M8

| Specification        | Value                             |
|----------------------|-----------------------------------|
| Model                | RPLIDAR A1M8                      |
| Ranging Principle    | Triangulation-based laser ranging |
| Range                | 0.15 m – 12 m                     |
| Scan Rate            | 5.5 Hz (default)   up to 10 Hz    |
| Simulation Scan Rate | 4.7 Hz (verified in Gazebo)       |
| Angular Resolution   | 1° (360 samples per revolution)   |
| Field of View        | 360°                              |
| Distance Accuracy    | < 1% of measured distance         |
| Laser Wavelength     | 785 nm (Class 1 laser, eye-safe)  |
| Interface            | UART / USB                        |
| Operating Voltage    | 5V DC, 400 mA                     |
| Dimensions           | 98.5 × 70 × 60 mm                 |
| Weight               | 170 g                             |
| ROS2 Topic           | /scan (sensor_msgs/LaserScan)     |

## 4.2 Intel RealSense D435 Depth Camera

| Specification     | Value                                                              |
|-------------------|--------------------------------------------------------------------|
| Model             | Intel RealSense D435                                               |
| Depth Technology  | Active IR stereo                                                   |
| Depth Range       | 0.1 m – 10 m                                                       |
| Depth Resolution  | 640 × 480 @ 30 fps                                                 |
| Colour Resolution | 1920 × 1080 @ 30 fps                                               |
| Horizontal FOV    | 87° (1.5184 rad — simulated)                                       |
| Vertical FOV      | 58°                                                                |
| Depth Accuracy    | < 2% at 2 m                                                        |
| Interface         | USB 3.1 Type-C                                                     |
| Operating Voltage | 5V, max 900 mA                                                     |
| Dimensions        | 90 × 25 × 25 mm                                                    |
| Weight            | 72 g                                                               |
| Mounting Position | Front of robot at z = 0.100 m from base_link                       |
| ROS2 Topics       | /depth_camera/points (PointCloud2), /depth_camera/scan (LaserScan) |

### 4.3 IMU Sensor (MPU6050)

| Specification          | Value                                               |
|------------------------|-----------------------------------------------------|
| Model                  | MPU-6050 (6-DOF IMU)                                |
| Gyroscope Range        | $\pm 250$ / $\pm 500$ / $\pm 1000$ / $\pm 2000$ °/s |
| Accelerometer Range    | $\pm 2g$ / $\pm 4g$ / $\pm 8g$ / $\pm 16g$          |
| Update Rate            | 100 Hz (simulated)                                  |
| Simulation Update Rate | 93 Hz (verified in Gazebo)                          |
| Interface              | I2C / SPI                                           |
| Operating Voltage      | 3.3V / 5V                                           |
| Noise Density (Gyro)   | 0.005 °/s/ $\sqrt{\text{Hz}}$                       |
| Mounting Position      | z = 0.068 m from base_link                          |
| EKF Usage              | Fused with /odom to produce /odometry/filtered      |
| ROS2 Topic             | /imu (sensor_msgs/Imu)                              |

## 5. Multi-Sensor Fusion

The Falco Rover fuses data from all three sensors at two distinct levels of the navigation stack:

### 5.1 Level 1 — State Estimation (EKF)

The `robot_localization` package runs an Extended Kalman Filter that fuses:

- Wheel odometry (`/odom`) — provides linear velocity and relative position from encoder pulses
- IMU data (`/imu`) — provides angular velocity and linear acceleration, correcting odometry drift during rotation
- The EKF output (`/odometry/filtered`) is a smoothed, drift-corrected pose estimate that AMCL and Nav2 use as the base for further localisation.

### 5.2 Level 2 — Obstacle Detection (Dual Costmap Sources)

Both the local and global costmaps subscribe to two independent observation sources:

- LiDAR scan (`/scan`) — 360° coverage, range up to 3.5 m, primary obstacle source
- Depth camera scan (`/depth_camera/scan`) — 180° forward arc, captures low obstacles (0–50 cm height) that the 2D LiDAR plane misses

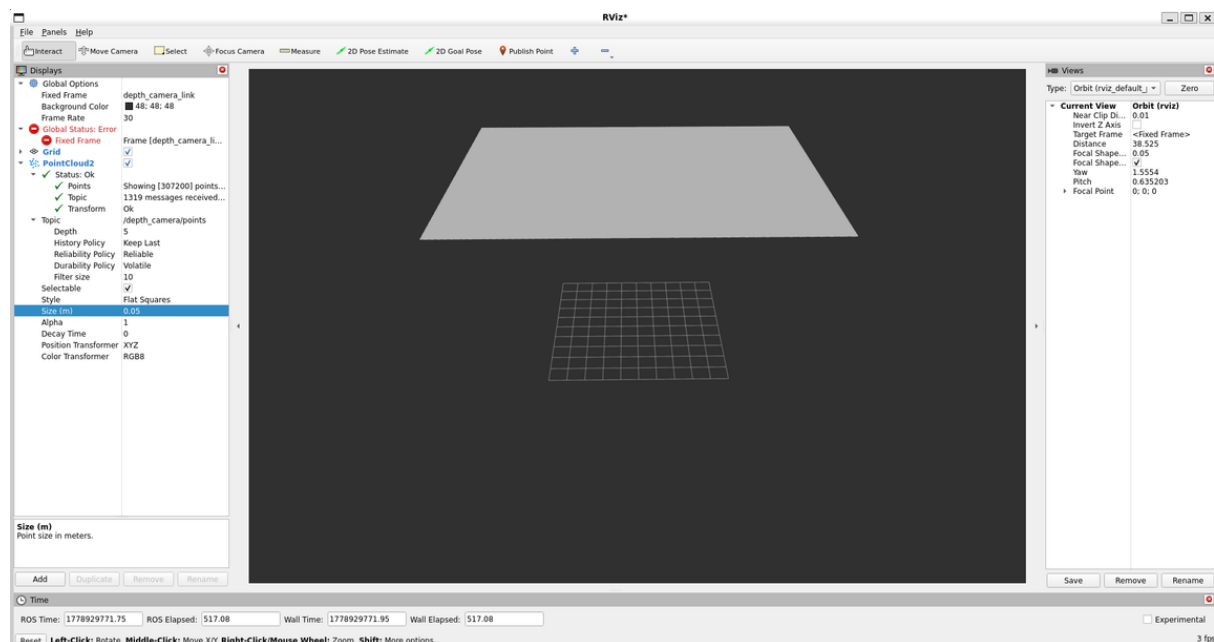


Figure 2: Intel RealSense D435 Depth Camera Point Cloud (307,200 points)

The `pointcloud_to_laserscan` node converts the 3D RealSense pointcloud into a 2D scan slice at floor level, enabling Nav2 to treat it as a second laser scanner. This means obstacles invisible to the LiDAR (low objects, carpet edges, ramps) are detected and marked in the costmap by the depth camera.

### 5.3 Level 3 — Localisation (AMCL + EKF)

AMCL uses LiDAR scan data to match current sensor readings against the pre-built map, producing a probabilistic pose estimate. This is combined with the EKF-filtered odometry via the TF tree, giving the robot three complementary position estimates fused through the TF broadcast chain: map → odom → base\_footprint → base\_link.

## 6. ROS2 Node Architecture

The following describes the complete node graph of the Falco Rover system as verified by `ros2 node list` during simulation. The full `rqt_graph` is included in the appendix.

### 6.1 Active Nodes

| Node                            | Function                                                          |
|---------------------------------|-------------------------------------------------------------------|
| /gazebo                         | Gazebo physics simulation engine with sensor plugins              |
| /gazebo_ros_lds_lfcd_controller | LiDAR plugin — publishes /scan at 4.7 Hz                          |
| /depth_camera_controller        | Depth camera plugin — publishes /depth_camera/points at 30 Hz     |
| /imu_plugin                     | IMU plugin — publishes /imu at 100 Hz                             |
| /diff_drive                     | Differential drive plugin — subscribes /cmd_vel, publishes /odom  |
| /robot_state_publisher          | Publishes URDF to /robot_description, broadcasts static TF frames |
| /pointcloud_to_laserscan_node   | Converts /depth_camera/points → /depth_camera/scan                |
| /ekf_filter_node                | Fuses /odom + /imu → /odometry/filtered                           |
| /amcl                           | Localises robot on map using /scan, publishes /amcl_pose          |
| /map_server                     | Loads saved map (my_map.yaml), publishes /map                     |

|                                |                                                                          |
|--------------------------------|--------------------------------------------------------------------------|
| /planner_server                | Global path planning (A*) on global_costmap                              |
| /controller_server             | Local obstacle avoidance (DWB), publishes /cmd_vel                       |
| /bt_navigator                  | Behaviour tree orchestrating the full navigation pipeline                |
| /local_costmap/local_costmap   | Rolling 3m×3m costmap from /scan + /depth_camera/scan                    |
| /global_costmap/global_costmap | Full-environment costmap from map + /scan + /depth_camera/scan           |
| /behavior_server               | Recovery behaviours: spin, backup, wait                                  |
| /velocity_smoother             | Smooths velocity commands before publishing to /cmd_vel                  |
| /voice_commander (custom)      | Subscribes to navigation topics, announces status via espeak             |
| /indicator_node (custom)       | Subscribes to navigation topics, publishes LED status (GREEN/RED/YELLOW) |
| /rviz2                         | Visualisation — map, costmap, path, LaserScan, depth camera image        |

## 6.2 Key Topics

| Topic                | Message Type              | Flow                                    |
|----------------------|---------------------------|-----------------------------------------|
| /scan                | LaserScan                 | LiDAR → AMCL, costmaps                  |
| /depth_camera/points | PointCloud2               | RealSense → pointcloud_to_laserscan     |
| /depth_camera/scan   | LaserScan                 | pointcloud_to_laserscan → costmaps      |
| /imu                 | Imu                       | IMU plugin → ekf_filter_node            |
| /odom                | Odometry                  | diff_drive → ekf_filter_node            |
| /odometry/filtered   | Odometry                  | EKF → Nav2, TF tree                     |
| /map                 | OccupancyGrid             | map_server → AMCL, global_costmap       |
| /amcl_pose           | PoseWithCovarianceStamped | AMCL → bt_navigator, Nav2               |
| /cmd_vel             | Twist                     | controller_server → diff_drive motors   |
| /goal_pose           | PoseStamped               | User (RViz) → bt_navigator              |
| /tf, /tf_static      | TFMessage                 | map → odom → base_footprint → base_link |



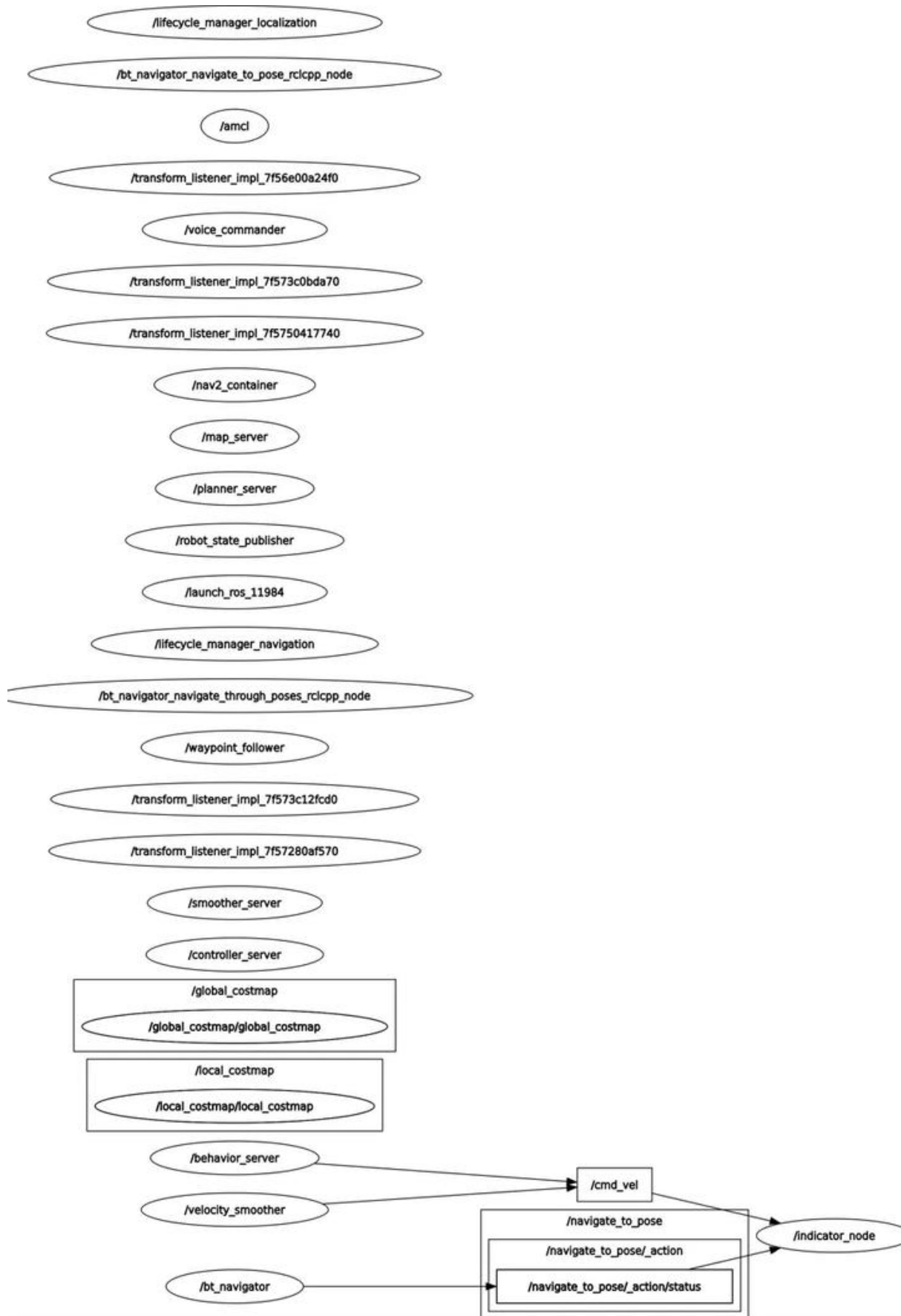


Figure 5: ROS2 Node Architecture — Key nodes and connections

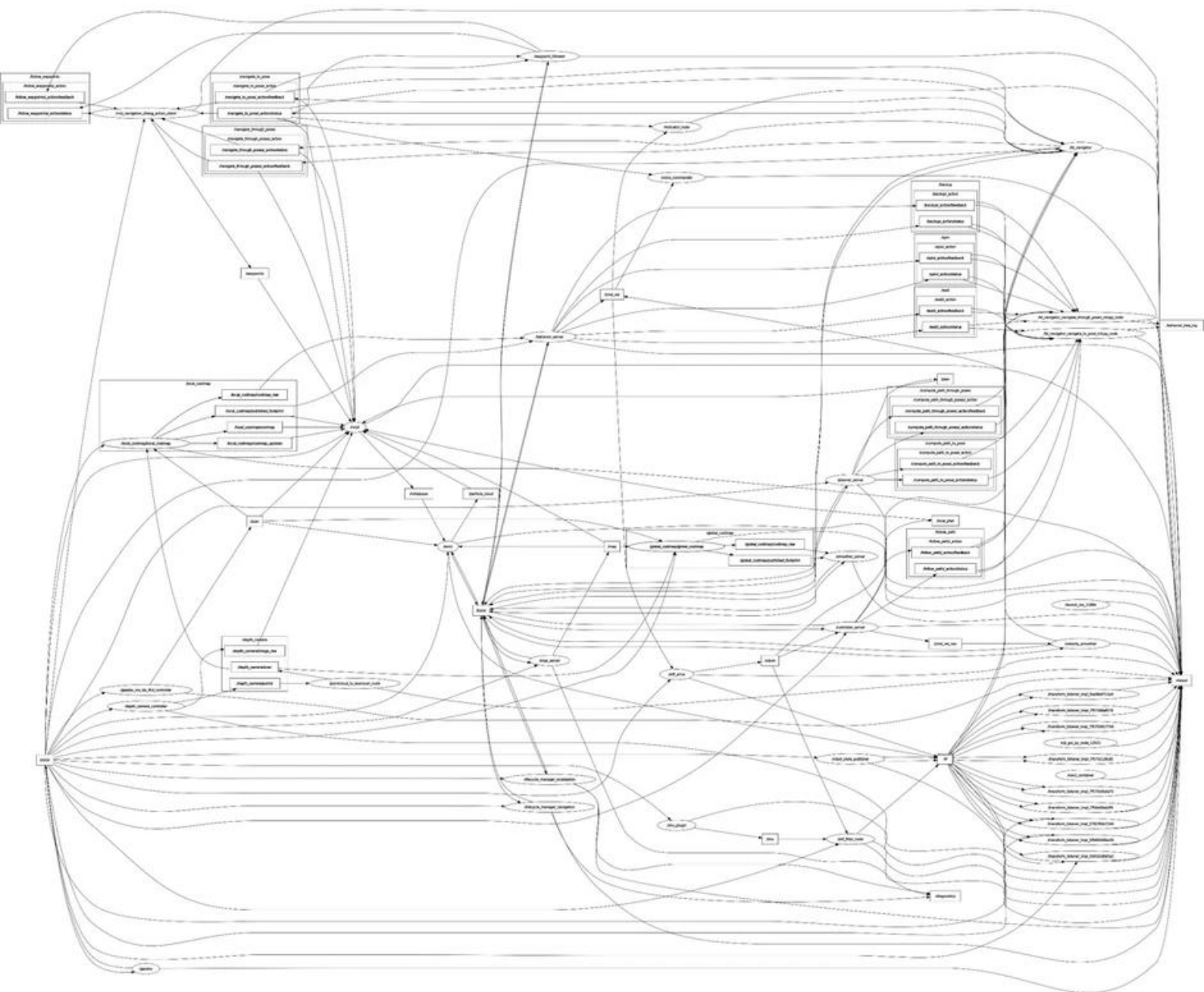


Figure 6: Complete ROS2 Node Graph (rq\_t\_graph)

## 7. Assumptions

- The RP LiDAR model assumed is the A1M8 — the most common entry-level LiDAR in ROS robotics. The Gazebo simulation scan rate (4.7 Hz) closely matches the A1M8 specification (5.5 Hz).
- The depth camera is assumed to be the Intel RealSense D435 — simulated via the `libgazebo_ros_camera.so` plugin with 87° horizontal FOV and 0.1–10 m range.
- The IMU is assumed to be the MPU-6050 6-DOF sensor — simulated via the `libgazebo_ros_imu_sensor.so` plugin at 100 Hz.
- The user provides ONE initial pose estimate via RViz (2D Pose Estimate) before autonomous navigation begins. This is standard practice as GPS is unavailable.
- The user provides ONE navigation goal via RViz (Nav2 Goal). After this, the robot operates with full autonomy — no further user input is required.
- The environment is mapped once using SLAM before autonomous navigation begins. The saved map is assumed to accurately represent the indoor environment.
- The indoor environment is a closed, bounded space with static obstacles. Dynamic obstacles are handled by the local costmap in real-time.
- The RPi computational role is simulated by the host CPU running ROS2 Humble on Ubuntu 22.04 via WSL2.

## 8. Custom Contributions

The following components were built from scratch as original contributions, not simply running pre-existing packages:

### 8.1 Custom URDF — `falco_rover`

A complete robot description file built by extending the TurtleBot3 Waffle Pi model with:

- A dedicated `depth_camera_link` with correct physical mounting position ( $x=0.073$ ,  $y=-0.011$ ,  $z=0.100$  from `base_link`)
- A Gazebo depth camera plugin (`libgazebo_ros_camera.so`) configured for 640×480 resolution, 30 Hz, 0.1–10 m range
- LiDAR plugin — 360 samples, 5 Hz, 0.12–3.5 m range
- IMU plugin — 100 Hz
- Differential drive plugin with correct wheel separation (0.287 m) and wheel diameter (0.066 m)

### 8.2 Custom ROS2 Workspace — `falco_rover_ws`

A complete ROS2 `ament_python` package (`falco_rover_bringup`) containing:

- `falco_navigation.launch.py` — launches Gazebo, Nav2, EKF, `pointcloud_to_laserscan`, `voice_commander`, and `indicator` in a single command
- `ekf.yaml` — EKF sensor fusion configuration specifying which sensor inputs to fuse and their noise parameters
- `nav2_params.yaml` — complete Nav2 parameter file tuned for the `falco_rover`, including dual observation sources (`scan` + `depth_scan`) in both local and global costmaps

### 8.3 `voice_commander.py` — Custom Node

A Python ROS2 node that subscribes to navigation action feedback topics and uses the `espeak` text-to-speech engine to announce navigation events in real-time: goal accepted, robot moving, goal reached, navigation failed, recovery activated.

### 8.4 `indicator.py` — Custom Node

A Python ROS2 node that subscribes to navigation status topics and publishes LED status to the terminal (simulating physical LED indication): GREEN when goal is reached, YELLOW when robot is actively navigating, RED when idle or failed.

## 8.5 Depth Camera Integration into Nav2

A key original contribution is the integration of the RealSense depth camera into the Nav2 navigation stack via the `pointcloud_to_laserscan` pipeline. This required:

- Adding the `pointcloud_to_laserscan` node to the launch file with correct topic remappings
- Configuring both `local_costmap` and `global_costmap` to subscribe to two observation sources (`scan` and `depth_scan`)
- Tuning the height slice parameters to capture ground-level obstacles
- Fixing the URDF to include the depth camera Gazebo plugin — not present in the base TurtleBot3 model

## 9. Simulation Results

### 9.1 Verified Sensor Outputs

| Topic                | Expected Rate       | Measured Rate | Status     |
|----------------------|---------------------|---------------|------------|
| /scan                | 5.5 Hz (A1M8 spec)  | 4.87 Hz       | ✓ Verified |
| /imu                 | 100 Hz              | 93 Hz         | ✓ Verified |
| /depth_camera/points | 30 Hz               | Publishing    | ✓ Verified |
| /depth_camera/scan   | Derived from points | Publishing    | ✓ Verified |
| /odom                | Continuous          | Publishing    | ✓ Verified |
| /odometry/filtered   | Continuous          | Publishing    | ✓ Verified |

### 9.2 Navigation Performance

- AMCL localisation: converged successfully after initial 2D pose estimate
- Global path planning: A\* path computed successfully for all test goals
- Local obstacle avoidance: DWB planner navigated around all 9 obstacles in real-time
- Goal completion: Feedback status reached 'reached' on all test runs
- Indicator node: LED GREEN confirmed on goal completion in terminal
- Voice commander: Announcements triggered correctly at all navigation events
- Recovery behaviours: Available and configured (spin, backup, wait)
- Depth camera: pointcloud\_to\_laserscan verified publishing /depth\_camera/scan to costmaps

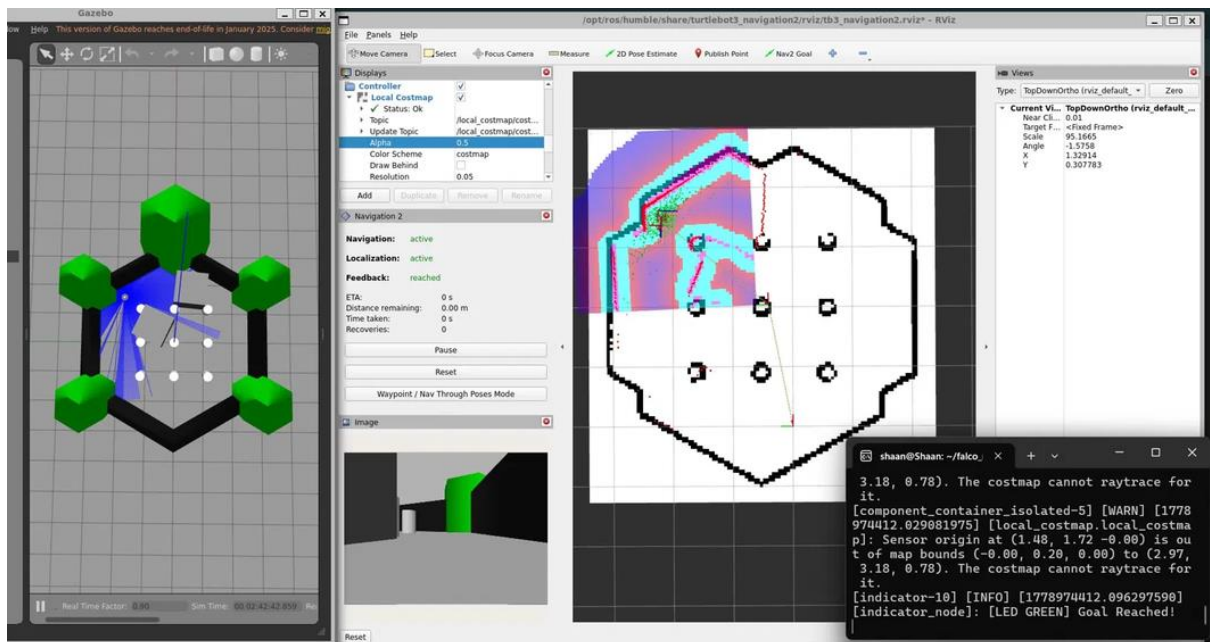


Figure 3: Autonomous Navigation — Robot reaching goal (Feedback: Reached)

### 9.3 Node Verification

All required nodes confirmed active via ros2 node list during simulation run:

| Node Category | Nodes Verified                                                                      |
|---------------|-------------------------------------------------------------------------------------|
| Simulation    | /gazebo, /robot_state_publisher                                                     |
| Sensors       | /gazebo_ros_lds_lfcd_controller, /depth_camera_controller, /imu_plugin, /diff_drive |
| Sensor Fusion | /ekf_filter_node, /pointcloud_to_laserscan_node                                     |
| Localisation  | /amcl, /map_server                                                                  |

|              |                                                                      |
|--------------|----------------------------------------------------------------------|
| Navigation   | /bt_navigator, /planner_server, /controller_server, /behavior_server |
| Costmaps     | /local_costmap/local_costmap, /global_costmap/global_costmap         |
| Custom Nodes | /voice_commander, /indicator_node                                    |

```

WARNING: Be aware that there are nodes in the graph that share an exact name, which can have unintended side effects.
/anc1
/behavior_server
/bt_navigator
/bt_navigator.navigate_through_poses_rclcpp_node
/bt_navigator.navigate_to_pose_rclcpp_node
/controller_server
/depth_camera_controller
/diff_drive
/ekf_filter_node
/ekf_filter_node
/gazebo
/gazebo_ros_lds_lfcd_controller
/global_costmap/global_costmap
/ima_plugin
/indicator_node
/indicator_node
/launch_ros_11984
/lifecycle_manager_localization
/lifecycle_manager_navigation
/local_costmap/local_costmap
/map_server
/mav2_container
/planner_server
/pointcloud_to_laserscan_node
/robot_state_publisher
/robot_state_publisher
/rqt_gui_py_node_13521
/rviz2
/rviz_navigation_dialog_action_client
/smooth_server
/transform_listener_impl_56032c6fe5a0
/transform_listener_impl_5761f58e7240
/transform_listener_impl_5ea0bbf7c1e0
/transform_listener_impl_5f066618ba30
/transform_listener_impl_7f56e00a24f0
/transform_listener_impl_7f57280af570
/transform_listener_impl_7f573c0bda70
/transform_listener_impl_7f573c12fcd0
/transform_listener_impl_7f5750417740
/velocity_smoother
/voice_commander
/voice_commander
/waypoint_follower
shaan@shaan:~$

```

Figure 4: Active ROS2 Nodes verified via `ros2 node list`



## 10. Conclusion

The Falco Rover successfully demonstrates a complete autonomous indoor navigation system operating without GPS. The system fuses data from three sensors — RP LiDAR A1M8, Intel RealSense D435, and IMU — at two levels of the navigation stack: EKF-based state estimation and dual-source costmap obstacle detection.

The key innovation of this implementation is the depth camera integration into Nav2's obstacle layer via the `pointcloud_to_laserscan` pipeline. This gives the rover the ability to detect low obstacles that a 2D LiDAR plane would miss, making the navigation significantly more robust in real indoor environments.

The system requires only two initial inputs from the user (pose estimate and navigation goal) and operates fully autonomously thereafter, including self-recovery from stuck situations via the Nav2 behaviour server.

The simulation in Gazebo with the custom `falco_rover` model validates the complete navigation pipeline from sensor data through to motor commands, with all nodes verified via `ros2 topic hz` and the `rqt_graph` node diagram.